Decision Trees

Tree structure, training, entropy, Gini impurity, and the full sklearn workflow.

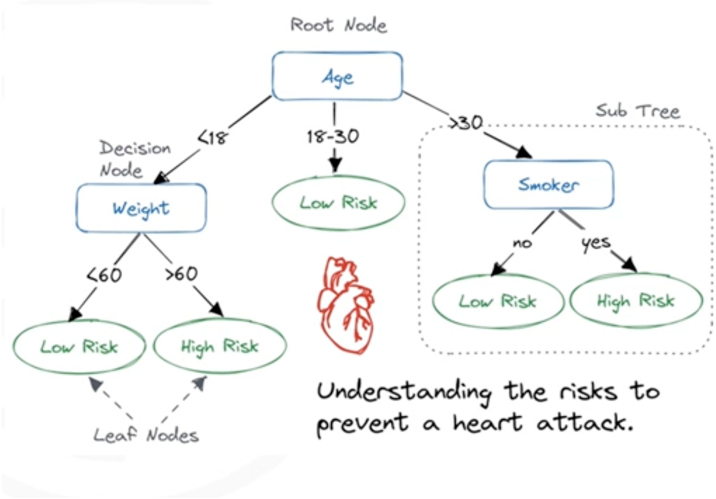
Tree Structure

Rule-based tree

DESCRIPTION

A decision tree is a flowchart-like graph that maps feature conditions to class predictions. It has three kinds of nodes: • Root node — the very first split; uses the most informative feature in the whole dataset. • Decision nodes — internal nodes that test one feature and branch left or right based on the result. • Leaf nodes — terminal nodes that hold the final predicted class (no more splits below them). Each path from root to leaf is a human-readable rule, e.g. "If Na_to_K > 14.8 → drugY".

EXAMPLE



Example decision tree — root node, decision nodes, and leaf nodes.

CODE

```
# Print a text representation of the tree — shows root, decision, and leaf nodes
from sklearn.tree import export_text

tree_rules = export_text(
    drugTree,
    feature_names=['Age', 'Sex', 'BP', 'Cholesterol', 'Na_to_K'],
)
print(tree_rules)

# Example output:
# |--- Na_to_K <= 14.82
# |   |--- BP <= 0.50
# |   |   |--- class: drugX      <- leaf node
# |   |--- BP > 0.50
# |   |   |--- class: drugA      <- leaf node
# |--- Na_to_K > 14.82
# |   |--- class: drugY          <- leaf node
```

Training a Decision Tree

Rule-based tree

DESCRIPTION

Training builds the tree top-down by greedily picking the best split at each node: 1. Start with the root node containing ALL training samples. 2. Search every feature for the threshold that best separates the classes (measured by information gain or Gini impurity). 3. Split the node's data into two child nodes on that threshold. 4. Repeat recursively for each child until a stopping criterion is met (e.g. max_depth, pure leaf, or too few samples). The result is a tree where each internal node asks exactly one yes/no question about one feature.

CODE

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split

X_trainset, X_testset, y_trainset, y_testset = train_test_split(
    X, y, test_size=0.3, random_state=3
)

# criterion="entropy" uses information gain to choose splits
# max_depth limits tree size to avoid overfitting
drugTree = DecisionTreeClassifier(criterion="entropy", max_depth=4)
drugTree.fit(X_trainset, y_trainset)
```

Entropy & Information Gain

Rule-based tree

DESCRIPTION

Entropy measures the impurity (disorder) of a node's class distribution: $H = -\Sigma\ p_i \cdot \log_2(p_i)$ Information gain is the drop in entropy after a split — the algorithm picks whichever feature maximises this drop: IG = H(parent) − weighted_avg(H(children)) Higher information gain → the split tells us more → better feature to split on.

SAMPLE DATA

Entropy	Meaning	Example split
0	Pure — all samples same class	10/10 drugA
~0.5	Low — mostly one class	8/10 drugA, 2/10 drugB
~1.0	High — classes roughly equal	5/10 drugA, 5/10 drugB
Max	Completely mixed	Equal spread across all classes

CODE

```
# sklearn computes entropy internally; you can inspect it via the tree object
import numpy as np

def entropy(labels):
    classes, counts = np.unique(labels, return_counts=True)
    probs = counts / counts.sum()
    return -np.sum(probs * np.log2(probs + 1e-9))

print("Root entropy:", entropy(y_trainset))
# After fitting, each node's impurity is stored in:
print(drugTree.tree_.impurity)   # array of entropy values per node
```

Gini Impurity

Rule-based tree

DESCRIPTION

Gini impurity is an alternative splitting criterion to entropy. It measures the probability of mislabelling a randomly chosen sample: Gini = 1 − Σ p_i^2 • Gini = 0 → perfectly pure node (all one class). • Gini = 0.5 → maximally impure for a binary problem. Gini is slightly faster to compute (no logarithm) and tends to produce similar trees to entropy in practice. Use criterion='gini' in sklearn.

CODE

```
# Switch between criteria — results are usually very close
tree_gini    = DecisionTreeClassifier(criterion="gini",    max_depth=4)
tree_entropy = DecisionTreeClassifier(criterion="entropy", max_depth=4)

tree_gini.fit(X_trainset, y_trainset)
tree_entropy.fit(X_trainset, y_trainset)

from sklearn import metrics
for name, model in [("Gini", tree_gini), ("Entropy", tree_entropy)]:
    preds = model.predict(X_testset)
    print(f"{name} accuracy: {metrics.accuracy_score(y_testset, preds):.3f}")
```

Encoding Categorical Features

Categorical encoding

DESCRIPTION

Decision trees in sklearn only accept numeric inputs. Categorical columns (e.g. Sex = 'M'/'F', BP = 'LOW'/'NORMAL'/'HIGH') must be converted to integers before fitting. LabelEncoder assigns each unique string a stable integer (alphabetical order). It's fine for ordinal or small-cardinality columns used in a tree — the tree only checks thresholds, so the exact integer values don't need to be meaningful.

CODE

```
from sklearn.preprocessing import LabelEncoder

label_encoder = LabelEncoder()

# Encode each categorical column in-place
my_data['Sex']         = label_encoder.fit_transform(my_data['Sex'])
my_data['BP']          = label_encoder.fit_transform(my_data['BP'])
my_data['Cholesterol'] = label_encoder.fit_transform(my_data['Cholesterol'])

# Inspect the dataset after encoding
my_data.info()
print(my_data.head())
```

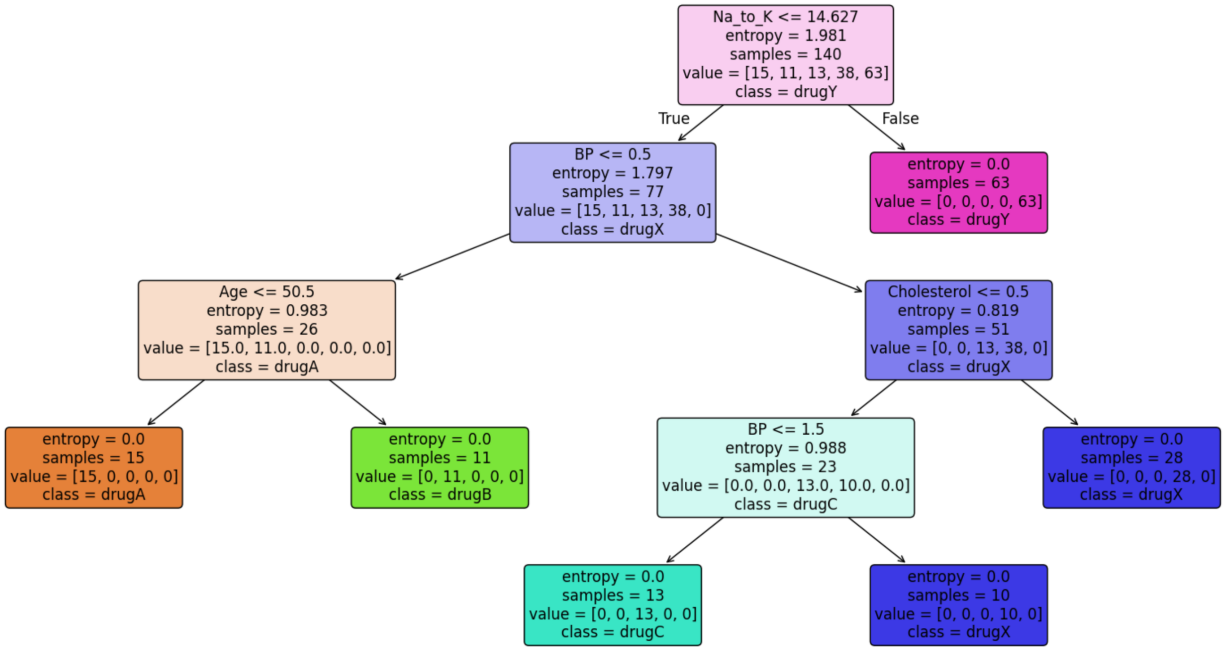
Visualising the Tree

Rule-based tree

DESCRIPTION

plot_tree() renders the fitted tree as a graphical diagram. Each node shows the split condition, sample count, impurity, and majority class. Setting filled=True colours nodes by their majority class — making leaf nodes instantly readable at a glance. Tips: use figsize to give the canvas enough room for deep trees, and feature_names / class_names to replace the default x[0], x[1] placeholders with your actual column and class names.

EXAMPLE



Output of plot_tree() — coloured nodes show majority class at each split.

CODE

```
from sklearn.tree import plot_tree
import matplotlib.pyplot as plt

plt.figure(figsize=(20, 10))
plot_tree(
    drugTree,
    feature_names=['Age', 'Sex', 'BP', 'Cholesterol', 'Na_to_K'],
    class_names=drugTree.classes_,
    filled=True,      # colour nodes by majority class
    rounded=True,
    fontsize=12,
    proportion=False,
)
plt.tight_layout()
plt.show()
```